

Recursion Scheme の紹介

λ Kansai in Winter 2026 - 2026-01-17

グミ (@GummyCandy1206)

はじめに

はじめに

動機

- LT 会での発表の経験がほぼないので、発表したい
- 関数型プログラミングに興味がある



λ Kansai in Winter 2026 が凄くちょうどよい！

本スライドで紹介すること

- **Recursion Scheme** (再帰関数から再帰に関する部分だけを抜き出して抽象化したもの)
- 本スライドでは特に `foldr` の一般化である **catamorphism** について紹介する

個人的おもしろポイント

- 圏論をプログラミングに応用している例である
 - ▶ 圏論の言葉を用いることでシンプルに説明できる
 - ▶ 可換になるように定義すると、正しいコードになる
 - ▶ 双対や射の合成によって、新たなアルゴリズムが生まれる

仮定する事前知識

- Haskell の基礎知識 (Functor、型クラスなど)
- 圏論の基礎知識 (関手、始対象など)

foldr

foldr

たし算やかけ算、文字列の結合などの二項演算を各要素に順番に適用する関数

```
foldr (+) 0 [1,2,3] == 6 -- すべてたす
foldr (*) 1 [2,3,4] == 24 -- すべてかける
foldr (++) "" ["Hello ", "World!"] == "Hello World!" -- すべて結合する
```


計算の流れ

```
foldr (+) 0 [1,2,3]
1 + foldr (+) 0 [2,3]
1 + (2 + foldr (+) 0 [3])
1 + (2 + (3 + foldr (+) 0 []))
1 + (2 + (3 + 0))
1 + (2 + 3)
1 + 5
6
```

foldr は**再帰的**に計算する。

リストについてももう少し詳しく

リスト型は以下のように定義されている。

```
data List a = [] | a : List a
```

リスト型は**再帰的**に定義されている。

リスト[1,2,3]の表記はシンタックスシュガーなので、定義に戻したものを確認する。

```
[1,2,3] == 1 : (2 : (3 : []))
```

ここまでのまとめ

- `foldr` は再帰的に計算する
- リスト型は再帰的に定義されている

再帰と再帰以外に分離する

ListF

List の再帰的な構造を再帰に関する部分とそれ以外の部分に分ける。

```
data List a = [] | a : List a
```

-- ↓ 再帰とそれ以外に分解

```
newtype Fix f = Fix { unFix :: f (Fix f) } -- 再帰に関する部分
```

```
data ListF a b = Nil | Cons a b deriving (Functor) -- 再帰以外の部分
```

Fix (**ListF** a)が **List** a に対応している。

List と ListF の対応

[1,2,3]は ListF では次のように書ける。

```
Fix (Cons 1 (Fix (Cons 2 (Fix (Cons 3 (Fix Nil))))))
```

nil = Fix Nil、cons a b = Fix (Cons a b)とすれば、リストとの対応がわかりやすい。

```
1 :      (2 :      (3 :      [] )) :: List a  
1 `cons` (2 `cons` (3 `cons` nil)) :: Fix (ListF a)
```

catamorphism の定義

`foldr` の一般化である `catamorphism` を定義する。以降、`cata` と表記する。

```
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
```

```
cata :: Functor f => (f a -> a) -> Fix f -> a  
cata f = f . fmap (cata f) . unFix
```

これだけ見ても `foldr` の一般化であることは分かりづらい。証明も少し面倒なので、計算例をいくつか見るだけにしておく。

リストの和に対応する例

`foldr (+) 0 [1,2,3]`に対応する例は以下の通り。

```
newtype Fix f = Fix { unFix :: f (Fix f) }

cata :: Functor f => (f a -> a) -> Fix f -> a
cata f = f . fmap (cata f) . unFix

data ListF a b = Nil | Cons a b deriving (Functor)

listSumAlgebra :: ListF Int Int -> Int
listSumAlgebra Nil      = 0
listSumAlgebra (Cons a b) = a + b

main = do
  let xs = Fix (Cons 1 (Fix (Cons 2 (Fix (Cons 3 (Fix Nil)))))
  print (cata listSumAlgebra xs) -- 6
```


計算の流れ

```
cata listSumAlgebra (Fix Nil)
(listSumAlgebra . fmap (cata listSumAlgebra) . unFix ) (Fix Nil)
(listSumAlgebra . fmap (cata listSumAlgebra)) (unFix (Fix Nil))
(listSumAlgebra . fmap (cata listSumAlgebra)) Nil
listSumAlgebra (fmap (cata listSumAlgebra) Nil)
listSumAlgebra Nil
0
```

これは `foldr (+) 0 [] == 0` に対応している。

計算の流れ

$x :: a$ 、 $xs :: \text{Fix (ListF a)}$ とする。このとき、 Fix (Cons x xs) について計算してみる。

```
cata listSumAlgebra (Fix (Cons x xs))
(listSumAlgebra . fmap (cata listSumAlgebra) . unFix ) (Fix (Cons x xs))
(listSumAlgebra . fmap (cata listSumAlgebra)) (unFix (Fix (Cons x xs)))
(listSumAlgebra . fmap (cata listSumAlgebra)) (Cons x xs)
listSumAlgebra ((fmap (cata listSumAlgebra)) (Cons x xs))
listSumAlgebra (Cons x (cata listSumAlgebra xs))
x + cata listSumAlgebra xs
```

これは $\text{foldr (+) 0 (x:xs) == x + foldr (+) 0 xs}$ に対応している。

ここまでのまとめ

- `List` は再帰に関する部分の `Fix` とそれ以外の部分の `ListF` に分解できる。
- `foldr (+) 0` は再帰に関する部分の `cata` とそれ以外の部分の `listSumAlgebra` に分解できる。

圏論で見る cata

cata の定義

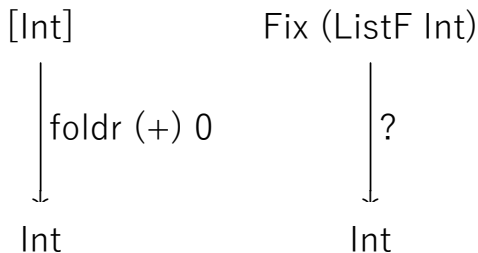
```
newtype Fix f = Fix { unFix :: f (Fix f) }  
  
cata :: Functor f => (f a -> a) -> Fix f -> a  
cata f = f . fmap (cata f) . unFix
```

cata の定義が初見時に理解できなかったなので、圏論を用いて大雑把に説明する。

可換図式

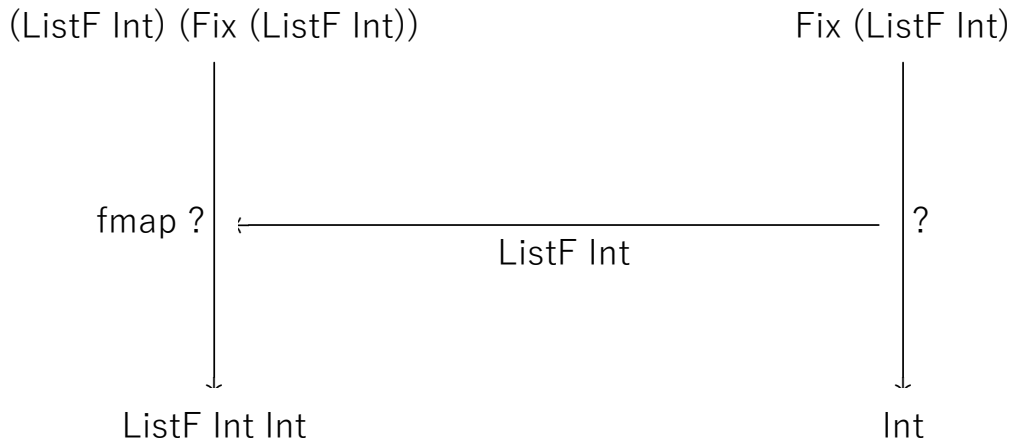
簡単のために、 $a = \text{Int}$ 、 $b = \text{Int}$ の場合を考える。

$\text{foldr } (+) \ 0 \ :: \ [\text{Int}] \rightarrow \text{Int}$ の catamorphism バージョンを考える。



可換図式

右の図式を関手 `ListF Int` で移す。



可換図式

unFix の定義より、以下の矢印がある。

```
newtype Fix f = Fix { unFix :: f (Fix f) }
```

$$\begin{array}{ccc} (\text{ListF Int}) (\text{Fix} (\text{ListF Int})) & \xleftarrow{\text{unFix}} & \text{Fix} (\text{ListF Int}) \\ \text{fmap ?} \downarrow & & \downarrow ? \\ \text{ListF Int Int} & & \text{Int} \end{array}$$

可換図式

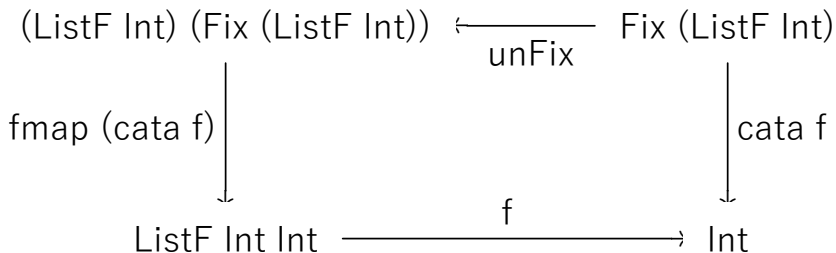
もし、`f :: ListF Int Int -> Int` という関数を与えると、
以下の図式を可換にする？ は次の等式を満たす。

`? == f . (fmap ?) . unFix`

$$\begin{array}{ccc} (\text{ListF Int}) (\text{Fix (ListF Int)}) & \xleftarrow{\text{unFix}} & \text{Fix (ListF Int)} \\ \text{fmap ?} \downarrow & & \downarrow ? \\ \text{ListF Int Int} & \xrightarrow{\text{f}} & \text{Int} \end{array}$$

可換図式

? を `cata f` と書けば、`cata f == f . (fmap (cata f)) . unFix` となるが、これは `cata f` の定義そのものの。



定義の説明としては不十分だと思うが、図式が可換になるように考えると定義が導かれる。

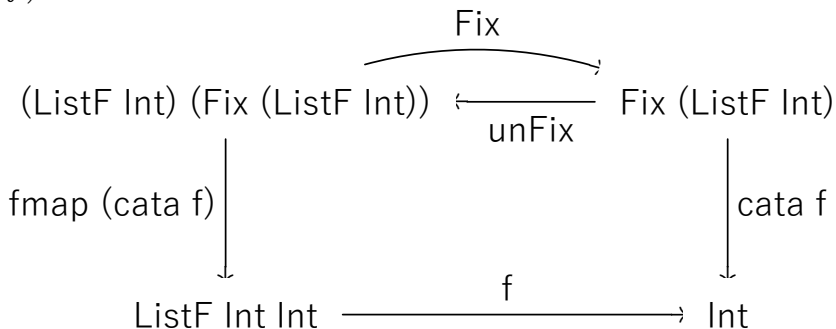
ここまでのまとめ

- 図式が可換になるように考えると、自然に `cata` の定義が得られる。

F 代数

F 代数について

関手 $F : C \rightarrow C$ に対して、対象 $a \in C$ と射 $f : Fa \rightarrow a$ の組 (a, f) を F 代数と呼ぶ。



ListF Int は関手なので、 $(\text{Fix} (\text{ListF Int}), \text{Fix})$ や (Int, f) は (ListF Int) 代数である。

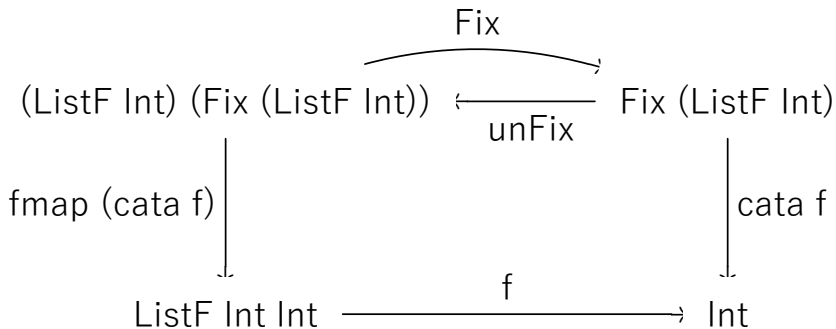
F 代数の射

F 代数の射 $\varphi : (a, f) \rightarrow (b, g)$ は圏 C の射 $\varphi : a \rightarrow b$ であって、 $\varphi \circ f = F\varphi \circ g$ を満たすもの。

$$\begin{array}{ccc} & \text{Fix} & \\ & \curvearrowright & \\ (\text{ListF Int}) (\text{Fix (ListF Int)}) & \xleftarrow{\text{unFix}} & \text{Fix (ListF Int)} \\ \text{fmap (cata f)} \downarrow & & \downarrow \text{cata f} \\ \text{ListF Int Int} & \xrightarrow{\quad f \quad} & \text{Int} \end{array}$$

$\text{cata } f$ は (ListF Int) 代数の射である。

F 代数の射



実は、 $(\text{Fix (ListF Int)}, \text{Fix})$ は (ListF Int) 代数の始対象となっている (F 始代数)。 cata f は始対象からの唯一の射である。

anamorphism

catamorphism の図式の双対は、anamorphism と呼ばれる (以降、**ana** と表記する)。

$$\begin{array}{ccc} (\text{ListF Int}) (\text{Fix (ListF Int)}) & \xrightarrow{\text{Fix}} & \text{Fix (ListF Int)} \\ \uparrow \text{fmap (ana f)} & & \text{ana f} \uparrow \\ \text{ListF Int Int} & \xleftarrow{f} & \text{Int} \end{array}$$

双対を考えることで、新たなアルゴリズムを得られる。

hylomorphism

`cata f` と `ana g` の合成を考えることで新たなアルゴリズムが得られる。

```
hylo f g = cata f . ana g
```

一度リストを作って、それを畳み込むイメージ。

(例) n 以下の整数のリストの積 (階乗)

metamorphism

`cata f` と `ana g` の合成の順番を逆にすることで別のアルゴリズムが得られる。

```
meta f g = ana f . cata g
```

一度集約してそれを展開するイメージ。

(例)

- AtCoder のある問題を metamorphism で考えた例 [連長圧縮したリスト上の分割\(Haskell\)](#)

その他にも様々な morphism が考えられている。

ここまでのまとめ

- `Fix (ListF Int)` は `ListF Int` 代数の始対象。
- `cata` は始対象からの唯一の射。
- 双対や射の合成で、新たなアルゴリズムが得られる。

全体のまとめ

まとめ

- Recursion Scheme は再帰とそれ以外に分離して考える
- catamorphism は `foldr` の一般化である
- 図式が可換になるように考えると、勝手に catamorphism の定義を導ける
- catamorphism は圏論 (F 代数) を用いるとシンプルに説明できる
- 双対や射の合成で新たなアルゴリズムを得られる



関数型プログラミングと圏論の両方の学習者にとって面白い例

ご清聴ありがとうございました。

補記

foldr の型

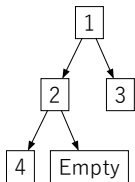
```
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
```

```
foldr (+) 0 [1,2,3] == 6
```

Foldable はリストの一般化。 リストとは限らないが、
toList :: Foldable t => t a -> [a]でリストにできる。 リ
ストっぽい型とすることができる。

リスト以外の型に対する foldr

二分木の値の和を求める例。



```
{-# LANGUAGE DeriveFoldable #-}  
data Tree a = Empty | Leaf a | Node (Tree a) a (Tree a) deriving Foldable  
  
tree = Node (Node (Leaf 4) 2 Empty) 1 (Leaf 3)  
  
main = do  
  print (foldr (+) 0 tree) -- 10 リストでなくても計算できる！
```


ListF a が Functor であることについて

ListF の定義の `deriving` 宣言で `Functor` であることを宣言している。

```
data ListF a b = Nil | Cons a b deriving (Functor)
```

初見時に混乱したので、少し補足しておく。 `ListF` が `Functor` になるのではなく、`ListF a` が `Functor` になる。

以下のような自明な `Functor` が導出される。

```
instance Functor (ListF a) where
  fmap f Nil      = Nil
  fmap f (Cons a b) = Cons a (f b)
```

リスト以外の例

リストだけだと `cata` のメリットが分かりづらい。他の例として、抽象構文木を文字列に変換する例を見る。

```
data ExprF x = LitF Int | AddF x x | MulF x x deriving (Functor)
```

```
type Expr = Fix ExprF
```

```
astToStr :: ExprF String -> String
```

```
astToStr (LitF n) = show n
```

```
astToStr (AddF a b) = "(" ++ a ++ " + " ++ b ++ ")"
```

```
astToStr (MulF a b) = "(" ++ a ++ " × " ++ b ++ ")"
```

```
ast :: Expr
```

```
ast = Fix (AddF (Fix (MulF (Fix (LitF 1)) (Fix (LitF 2))))  
              (Fix (AddF (Fix (LitF 3)) (Fix (LitF 4)))))
```

```
main = do
```

```
  putStrLn (cata astToStr ast) -- ((1 × 2) + (3 + 4))
```